CHEN XINGQIANG

杭州，中国
+86 150-0133-4125 ◇ joy6677@outlook.com ◇ github.com/chenxingqiang

求职意向

寻求**全职**职位：**AI 加速器编译器 / 系统软件 · 大语言模型推理系统 · 软硬件协同设计**。

交叉学科背景

理论物理与生物物理训练，贯通计算化学、分子动力学与计算机科学。竞赛：**全国大学生数学建模竞赛国家一等奖**（2012）、**北美大学生数学建模竞赛（MCM）一等奖**（2013）；**CCF 开源 GPU 创新生态赛 GPU 算子优化挑战赛二等奖**（2025）。

教育背景

生物物理学与凝聚态物理博士研究生（结业，学位答辩中） 2014 年 9 月 - 2018 年 7 月
厦门大学，物理与机电工程学院；导师方向：蛋白质动力学与 QM/MM 仿真

理论物理学学士 2010 年 9 月 - 2014 年 7 月
厦门大学，物理与机电工程学院

技能与项目

编程语言：C++、Python（主要）；CUDA、Java、Scala、Go。

编译器 / 工具链：MLIR、LLVM 后端与 ISA 适配、TVM、图划分与算子融合、成本建模、CodeGen、自动调优。

芯片软件 / 验证：Palladium Emulation、GDB 与仿真模型联调、固件与权重映射生成、算子级指标（Cycles/MACs）分析。

推理系统：Transformers、KV-Cache、量化、批处理与调度、Triton 内核、FlashAttention、CUPTI/Perfetto 性能剖析。

平台：PyTorch、TensorFlow、ONNX、Kubernetes；Spark、Flink、Hadoop。

开源与发布（默认位于 `github.com/chenxingqiang`，均可公开验证）：

- **TraceSmith**：跨平台 GPU Profiling & Replay 系统，覆盖 CUDA/ROCm/Metal/MACA/Ascend；C++ 核心 + Python 绑定，已发布至 PyPI。`TraceSmith`
- **YiRage 持久化内核（YPK）**：LLM 超级内核编译器与运行时，多后端（CUDA/CPU/MPS/Ascend/Triton）。`YiRage`
- **JAX-sklearn**：scikit-learn 兼容接口 + JAX 加速，13,058 项测试通过，PyPI 发布。`jax-sklearn`
- **Secret-Learn**：隐私保护机器学习库，覆盖联邦学习／秘密分享／分割学习，PyPI 发布。`Secret-Learn`
- **YouTube-DNN-RecSys**：YouTube 风格召回与排序的 TensorFlow + Spark 实现（60 stars）。`YouTube-DNN-RecSys`
- **Research CLI**：面向科研工作流的命令行 AI 工具，多模型供应商与 MCP 集成。`iechor-research/research-cli`
- **Questions-Gen**：数学竞赛题目生成（Qwen3-14B、GRPO、蒸馏），含 CLI 与 Hugging Face 权重。`GenQuestion`

工作经历

编译器 & 工具链专家，知存算力（Witmem） 杭州，中国
编译器 / *Palladium* 验证 *2025* 年 *12* 月 *10* 日 - 至今

- 负责存算芯片产品线编译器与 Palladium 仿真验证，覆盖模型编译、指令生成、芯片执行及 Emulation 缺陷定位。
- 完成 Palladium 软件 Bring-up：Compiler 输出在仿真器上运行，结合 GDB 在仿真模型、芯片与系统软件层做联合调试。
- 实现 **CIM-MLIR** 异构编译器：PyTorch/TensorFlow/ONNX 经 TOSA/StableHLO 映射至 CPU、GPU 与 CIM，含前端、图划分、硬件后端与运行时。
- 实现代价驱动图划分与 CIMHigh/Mid/Low 三级 IR；用 OR-Tools 分配 Die/L2/L0 资源，生成可烧录 **config.bin**、权重 **map.bin**，以及 GPU PTX 与 LLVM 主机代码。

- 实现跨设备运行时：分设备执行器、统一内存、计算与传输重叠及三级缓冲；提供 C++/Python 接口及延迟、利用率、传输开销统计。
- 实现 **GRC-PIM**：PIM 上基于 drift gating 与 GEMV 复用的残差校准及软判决纠错，投稿 *npj Unconventional Computing*（详见发表论文）。

高级 AI 编译器工程师，亿铸智能
杭州，中国

研发部门 *2022* 年 *9* 月 - *2025* 年 *12* 月

- 主导 **YiRage 持久化内核 (YPK)**：将端到端 LLM 解码编译为单个超级内核；在 Llama2-7B、单卡 A100 上相对 PyTorch eager 基线取得 **1.2×–6.7×** 端到端延迟收益（随 batch 与序列长度变化）。YiRage
- 实现融合算子（RMSNorm+Linear、GQA 等）与持久化执行模型，显著减少内核启动与中间张量往返；batch=1 decode 路径延迟降幅最大。
- 构建调度层并集成 Triton 与 FlashInfer 内核，支持多卡扩展（最高验证 8 卡）与 NVSHMEM 张量。
- 开发图编译与自动调优：内核参数搜索、KV-cache 友好内存布局，降低访存开销。
- 申请 **2 项**发明专利；相关能力交付 **2 个**数据中心客户 PoC。
- 负责公司 SoC 的 LLVM 后端 Target 与复杂指令集适配，推进编译器后端规范化与软件生态建设。
- 搭建模型转换与算子级评估流程：将 TVM、ONNX、MLCommons、TensorFlow/PyTorch 集成入 **YZlite**，统计 Cycles/MACs 等指标并输出多芯片 benchmark；开源工具包与文档 ReRAM-Labs/yzlite。

隐私保护算法工程师，蓝象智联
杭州，中国

IT Group *2022* 年 *1* 月 - *2022* 年 *9* 月

- 构建基于同态加密的联邦学习平台，支撑多家金融机构的跨域建模；在保持隐私约束下相对本地基线提升模型效果，并缩短训练迭代周期。
- 设计多方安全计算协议，用于银行风控模型的联合训练与推理，兼顾协议安全性与通信开销。
- 开源 **Secret-Learn**：sklearn 兼容 API × 联邦学习／秘密分享／分割学习三种模式，JAX 加速，并与 SecretFlow 集成，PyPI 发布。Secret-Learn
- 申请 **3 项**发明专利（均已公开，可检索）：基于秘密分享的纵向联邦线性支持向量分类方法 CN114925786A；随机森林训练方法及装置 CN114997345A；用于联邦学习的数据处理方法及装置 CN115130681A。

T7 算法工程专家，百图生科
北京，中国

算法工程组 *2021* 年 *9* 月 - *2021* 年 *12* 月

- 构建生物信息学工具链与数据平台，统一大规模生物数据的管理、检索与分析流程，支撑序列分析、结构预测与功能注释。
- 搭建蛋白质结构预测模型评估流程（含 AlphaFold 系列），为药物设计提供结构输入。
- 获 **2 项**软件著作权：生信工具自动化平台（登记号 2021SR2004464）、生物医药信息平台（登记号 2021SR1862397）。
- 开源 AlphaFold2 代码解析与学习资源（26 stars）。alphafold2-codecs

高级算法工程师，Yalla 集团 (纽交所: YALA)
杭州，中国

数据中心工程组 *2020* 年 *9* 月 - *2021* 年 *9* 月

- 重构 Yalla 聊天室推荐系统，引入多目标排序；A/B 实验中人均日活跃时长与 7 日留存均取得显著正向（留存 **42%** → **61%**）。
- 构建 Ludo 风控系统，结合实时行为特征与机器学习模型识别异常账号，显著降低恶意退款比例。
- 开发卡牌类游戏 AI（深度强化学习），用于人机对局与匹配填充，提升低峰时段对局体验与留存。

高级算法工程师，格家网络科技
杭州，中国

搜索推荐工程组 *2019* 年 *3* 月 - *2020* 年 *9* 月

- 设计多场景推荐架构，融合协同过滤、深度排序模型与商品知识图谱；线上实验 CTR 与 CVR 均取得两位数相对提升。
- 优化实时推荐链路：特征缓存与召回裁剪，P99 响应控制在 **50ms** 内，支撑大促期间峰值流量。
- 开源 YouTube-DNN-RecSys：YouTube 风格召回与排序的 TensorFlow + Spark 实现（60 stars）。YouTube-DNN-RecSys

机器学习工程师，便利蜂科技
北京，中国

数据工程组 *2017* 年 *3* 月 - *2019* 年 *3* 月

- 构建门店销量预测系统（覆盖 **50+** 门店），引入格点气象与时空特征，降低缺货与滞销率、提升库存周转。
- 开发用户与商品关联分析：在千万级行为数据上打通客户身份与购买记录，支撑个性化推荐。
- 设计商品视觉识别模型（**500+** SKU），用于快速结算场景，缩短收银与排队时间。

研究与实习

蛋白质分子动力学仿真与 QM/MM 模型
厦门大学，中国
(国家自然科学基金项目) *2014* 年 *9* 月 - *2017* 年 *3* 月

- 以 QM/MM 方法开展蛋白质动力学仿真与构象分析，成果发表于 Acta Cryst. D、PLoS ONE、Phys. Rev. E 等（见发表论文）。

液体活检 ct-DNA 试剂盒开发与 DNA 甲基化数据分析
乐普医疗，中国
(研发工程师实习) *2016* 年 *3* 月 - *2017* 年 *3* 月

- 在 TCGA/GEO 公开基因芯片数据上分析约 10,000 例样本，定位 Septin9 基因甲基化位点（结肠癌早期标志物），支撑临床诊断试剂盒的标志物筛选。

获奖与评审

- **国家一等奖**，全国大学生数学建模竞赛（CUMCM），2012 年。
- **一等奖**（Meritorious Winner），北美大学生数学建模竞赛（MCM/COMAP），2013 年。
- 全球**第 6 名**，国际太阳能十项全能竞赛（Solar Decathlon），2013 年。
- 第 **57** 名与第 **32** 名（共 1739 队），IEEE ICDM 全球气象 AI 挑战赛，2018 年。
- **二等奖**，第八届 CCF 开源创新大赛・开源 GPU 创新生态赛常规赛 S1 ・GPU 算子优化挑战赛（CCF/ODTC），2025 年。CCF 公示
- 评委，应用算法最佳实践初审，2021 年。

发表论文

- Chen X. GRC-PIM: A Drift-Gated, GEMV-Reusing Residual Calibrator for Soft-Decision Error Correction in Processing-in-Memory. *npj Unconventional Computing*（投稿中）。
- Cai Y, Su M, Ahmad A, Hu X, Sang J, Kong L, Chen X, Wang C, Shuai J, Han A. Conformational dynamics of the essential sensor histidine kinase WalK. *Acta Crystallographica Section D: Structural Biology*. 2017 Oct 1;73(10):793-803.
- Xia H, Chen X, Chen X, Cheng J, Liu Y, Chen X, Zhang Q, Zou G. Chiral discrimination of amino acid enantiomers based on different interactions with Cu2+. *Sensors and Actuators B: Chemical*. 2018 Jan 1;254:44-51.
- Ahmad A, Cai Y, Chen X, Shuai J, Han A. Conformational dynamics of response regulator RegX3 from Mycobacterium tuberculosis. *PloS one*. 2015 Jul 22;10(7):e0133389.
- Chen Y, Qi H, Li X, Cai M, Chen X, Liu W, Shuai J. Suppressing effect of C a 2+ blips on puff amplitudes by inhibiting channels to prevent recovery. *Physical Review E*. 2016 Aug 15;94(2):022411.

Open Source Architecture Appendix

Chen Xingqiang · github.com/chenxingqiang · draw.io source in docs/architecture/drawio/

TraceSmith

GPU Profiling & Replay
CUPTI / ROCm / Metal / MACA / Ascend

YiRage YPK

LLM megakernel compiler
12 backends + RuntimeFusion

JAX-sklearn

sklearn API + JAX proxy
CPU / GPU / TPU fallback

Secret-Learn

191 algos × FL / SL / SS
JAX + SecretFlow

YouTube-DNN

Two-tower recall
TensorFlow + Spark

Research CLI

AI CLI + academic tools
multi-provider / MCP

Questions-Gen

Qwen3-14B contest problems
Pretrain → GRPO → Distill

Each following page is a draw.io diagram exported to SVG, then appended to the resume PDF.

TraceSmith — Cross-platform GPU Profiling & Replay

github.com/chenxingqiang/TraceSmith · C++ core + pybind11 · PyPI: tracesmith

1. Workloads / producers

PyTorch / Kineto
kernel + runtime traces

TensorFlow / XLA
graph + device events

hooks

Custom CUDA / HIP
driver / runtime APIs

Multi-GPU cluster
NCCL collectives

2. User interface

CLI
profile / record / view
export / analyze / replay

Python API
pybind11 bindings
in-process capture

Distribution
pip install tracesmith
Apache-2.0

3. Capture backends (platform-specific, lock-free ingest)

NVIDIA CUPTI
nsys / kernel / memcpy

AMD ROCm
rocprof / HIP events

Apple Metal
xctrace / Instruments

MetaX MACA
MCPTI

Huawei Ascend
CANN / msprof

Linux eBPF
host syscall / driver

events

4. Core pipeline

SPSC ring buffer
lock-free ingest
<1% overhead

Common / stacks
event timeline
XRay / host stacks

export

SBT format
string interning
delta timestamps

State / timeline
GPU state machine
stream reconstruction

5. Replay, export, debug

Replay engine
full / partial / dry-run
stream scheduler
determinism checker

Perfetto
protobuf export
~85% smaller than JSON

Tracy profiler
live timeline view

GDB RSP
GPU breakpoints
trace-driven debug

Cluster sync
NTP / PTP / CUDA
NCCL tracking

Flow: workload → CLI/Python → platform backend → lock-free ring → SBT/timeline → replay or Perfetto/Tracy/GDB.

YiRage (YPK) — Multi-backend LLM Megakernel Compiler & Runtime

github.com/chenxingqiang/YiRage · based on Mirage (CMU) + multi-backend / RuntimeFusion extensions

1. Serving / embedding

vLLM
keeps schedule + KV
RF selects capsules

SGLang
decode-loop-erdecode
same FusionPlan cache

Python API
PersistentKernel
HardwareRegistry

UnifiedCompiler
graph + backend pick
Cython CoreBridge

2. RuntimeFusion

FusionPlan cache
search / profile results
per backend + shape

FusionCapsule
one decode-step kernel
selected each token

Device memory
persistent workspace
KV-friendly layout

cache

3. Compiler / search

Kernel graph IR
matmul / attn / norm
mark inputs / outputs

Superoptimize
hardware-aware search
15 candidate dims

Fingerprint
numeric verify
before accept

Autotune
grid / tile / pipeline
occupancy model

4. Threadblock operators + transpilers

MatMul
Tensor Core / Cube

Attention / GQA
Flash-style tiles

RMSNorm / SwiGLU
fused decode path

Triton codegen

NKI / MLIR

CUDA / HIP JIT

5. Backend registry (12 targets; factory + thread-safe singleton)

CUDA
Tensor Core
bank conflict

ROCm
HIP / MFMA

MPS
Apple Metal
unified mem

Ascend
AI Core / Cube
BiSheng

MACA
MetaX

TPU / XPU
NKI / Intel

MLIR / CPU
LLVM / SIMD

FPGA
experimental

Serving keeps scheduling/KV; compiler searches fused kernels; RuntimeFusion injects FusionCapsules into vLLM/SGLang decode.

JAX-sklearn (xlearn) — Drop-in sklearn with automatic JAX acceleration

github.com/chenxingqiang/jax-sklearn · 13,058 tests passed · pip install jax-sklearn

1. User code (zero API change)

import xlearn as sklearn
LinearRegression().fit(X, y)

Same estimators
cluster / decompose / preprocess

config_context()
force GPU / disable JAX

2. sklearn-compatible surface

Linear
LR / Ridge / Lasso
ElasticNet

Cluster
KMeans
282 tests

Decompose
PCA / TruncatedSVD
528 tests

Preprocess
Standard / MinMax

Others
native sklearn
via fallback

3. Runtime injection / proxy

__getattr__ lazy import
xlearn.linear_model …

JAX

_auto_jax_accelerate
replace estimator classes

EstimatorProxy
create JAX or original impl

Heuristics
fallback3 — KMeans > 1e6
PCA > 1e7 complexity

4. Acceleration + data

JAX kernels
JIT / vmap
universal mixins

Algo kernels
GP / MLP / SVM / tree

NumPy ↔ JAX
dtype / shape guards

Fallback
small data or error
→ original sklearn

5. Hardware abstraction (auto-select by problem size)

CPU
<10K samples
1.0–2.5×

NVIDIA GPU
10K–100K
5.5–8×

Google TPU
>100K
9.5–15×

Apple Metal
M1/M2/M3 beta
2–4×

Proxy decides JAX vs sklearn from dataset size, algorithm family, and configured platform; errors fall back silently.

Secret-Learn — Privacy-preserving ML (191 algorithms × FL / SL / SS)

github.com/chenxingqiang/Secret-Learn · sklearn API · JAX acceleration · SecretFlow devices

1. Applications

- **Healthcare** — hospitals keep records local
- **Finance** — cross-bank fraud / risk
- **IoT / edge** — federated sensors
- **Research** — multi-institution studies

2. sklearn-compatible API (191 estimators, 30+ categories)

- Linear / GLM
- Ensemble / trees
- SVM / neighbors
- Cluster / PCA
- NN / Naive Bayes
- Preprocess / select

3. Privacy mode router

- **FL — Federated**
  data stays on PYU
  ~~local JAX train~~
  HEU secure aggregate
  best: horizontal FL
- **SL — Split learning**
  model split by party
  ~~local JAX~~
  encrypted activations
  vertical / deep models
- **SS — Secret sharing**
  SPU MPC encrypt
  maximum privacy
  multi-party runtime

4. Intelligent generation + acceleration

- **Algorithm classifier**
  supervised? partial_fit?
  fit(x) vs fit(x,y)
- **Template migrator**
  auto-generate FL/SL/SS
  adapter from sklearn
- **JAX-sklearn core**
  local 5×+ speedup
  CPU / GPU / TPU
- **573 examples**
  one per algo × mode
  batch runners

5. SecretFlow device plane

- **PYU**
  plaintext local
  party compute
- **HEU**
  homomorphic
  secure aggregate
- **SPU**
  MPC / secret share
- **TEE**
  trusted execution
- **DP hooks**
  optional noise

Same estimator call; mode picks FL/SL/SS. FL keeps data local, SL splits the net, SS runs under MPC on SPU.

YouTube-DNN-RecSys — Two-tower recall (Covington et al.) in TF + Spark

github.com/chenxingqiang/YouTube-DNN-RecSys · Python TensorFlow · Scala Spark · 60 GitHub stars

1. Event / feature sources

Watch history
video / search ids

Context
device / geo / time

Continuous feats
age / counts

Candidate items
catalog embeddings

2. Dual implementation stacks

Python / TensorFlow
data2tfrecords.py
models/dnn.py (Estimator)
custom layers / TensorBoard
SavedModel export

Scala / Spark
FeatureBuilder.scala
BaseSparkLocal
PredictUserVector
ItemEmbeddingPredictor

3. Two-tower model

User tower
item-id embeddings
average watch history
concat continuous feats
MLP → user vector u

Matching
dot(u, v_i)
sampled softmax
next-watch objective

Item tower
shared item embedding
table v_i
optional side features

4. Train / serve

Train
TF Estimator
optional PS / workers
checkpoints

Eval
hold-out next-item
TensorBoard curves

Online recall
ANN / softmax top-k
user vec × item table

Artifacts
TFRecords
SavedModel / Spark vecs

History embeddings are mean-pooled into a user vector and scored against the item table — the paper's candidate-generation stage.

Research CLI — Command-line AI workflow with academic research tools

github.com/iechor-research/research-cli · TypeScript monorepo · Gemini CLI lineage + multi-provider + research modules

1. Interaction

packages/cli
Ink TUI · research.tsx

Slash commands
/research /paper /submit
/model config|select

Non-interactive
npx / CI scripts

VS Code companion
packages/vscode-ide

2. packages/core — orchestration

Prompt + context
GEMINI.md
file discovery

Session state
tool calls ~~compress~~
1M-token window

Policy / sandbox
~~tool permission~~
workspace isolation

Routing
provider + model
fallback chain

3. Multi-model providers

Gemini / Vertex

OpenAI

Anthropic

Baidu ERNIE

DeepSeek

Qwen / Groq / Kimi

4. Tools + MCP

Workspace tools
fs / shell / git

MCP servers
external capabilities

Search grounding
iEchor / web

SDK / a2a-server
programmatic embed

5. Academic research modules (packages/core/src/tools/research)

Bibliography
arXiv / PubMed / IEEE
BibTeX export

Writing
outline / grammar
LaTeX manager

Analysis
stats / citation net
experiment codegen

Submission
journal matcher
template / package

Prompt → core context → selected provider → tools/MCP/research modules → streamed terminal result.

Questions-Gen — Math competition problem generator (Qwen3-14B)

github.com/chenxingqiang/GenQuestion · Unsloth training · Hugging Face: xingqiang/* · pip install questions-gen

1. Training mixture

Historical contests
50% · CUMCM / olympiad

Conditional variants
30% · rewrite constraints

Innovative types
20% · new stems

data_preparer.py
tokenize / pack

2. Three-stage pipeline

Stage 1 — Pretrain
Qwen3-14B SFT
HF: stage1-fp-merged
base generation

Stage 2 — GRPO
8 problems / group
novelty constraint
policy optimization

score

Reward
0.4 difficulty
0.3 novelty
0.2 rigor
0.1 diversity

eval

Stage 3 — Distill
DeepSeek-R1 teacher
difficulty / logic / edu
final merged FP16

3. Validation (questions_gen/validation)

ModelValidator
single-model n tests

BatchValidator
algebra / calculus
geometry / number theory

QualityEvaluator
content / clarity
difficulty / originality

Scores
Final 0.847
teacher 4.2 / 5

4. Package / deploy

CLI
questions-gen
validate / batch
quality / compare

Python API
QuestionsGenTrainer
OllamaManager
HuggingFaceUtils

Hugging Face
stage1 / stage2 / final
xingqiang/*

Ollama
push-all
local generate API

Pretrain → GRPO with multi-dimensional rewards → R1 distillation → validate by category → publish HF weights and Ollama models.